Handoff 42: Keyboard tests, accessibility audit, screen reader sheet and cross-browser report (master run WE6)

Date: 2026-11-08 (America/Toronto)
Written by: Claude Code (implementer), in a separate git worktree
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request, not merged, not deployed. **Fixes a keyboard bug learners have today** (section 5). One new dev-only dependency, `axe-core`, approved by Lucas. No migration.

> **How to use this file:** upload it with `docs/plan/screen-reader-test-sheet.md`. The pull request diff is the source of truth (`build.md` section 14).

Starter prompt for the chat assistant

```
You are reviewing master run stage WE6 for Python Academy: QA-01a (keyboard-only tests, an
axe-core
audit, a manual screen reader sheet) and QA-03 (Firefox and WebKit as a non-blocking
report). Check
that the Esc fix in the code editor is the right pattern, that the tests really use only
the
keyboard, that the audit cannot pass by accident, and that the sheet would catch real
problems.
```

Run plan details

Field	Value
Stage	WE6
Base	`main`, at `091eecb`. No other PR needs to merge first
Part 2 boxes relied on	**None.** The new dependency was asked for under plan section 1.6 and approved by Lucas on 2026-11-08 ("u can make everything now")
Stages passed over to reach it	WE5's UI-02 to UI-04 (decision 29); UI-01 is merged. WE1 to WE4 need box B-W1

1. Step 0

`git fetch --all` and `gh pr list --state all`: #40 and #46 open. Worktree `../python-academy-qa`; the main folder was not switched. The repository was made public by Lucas during the stage, so Actions minutes for standard runners stopped being a cost.

2. Handoff record

Field	Value
Task	Keyboard-only tests, an automated accessibility audit, a manual screen reader sheet, and a non-blocking Firefox and WebKit run
Human goal	Find accessibility problems before learners do, and have a sheet for the checks only a person can make
Non-goals	Running the manual sheet (that needs people and real devices); making Firefox or WebKit blocking
Data and compatibility	None
Deployment status	Not requested. Once deployed, learners see the Esc fix and a reworded editor instruction

Files changed

File	Change
`frontend/app/code-editor.tsx`	Esc lets the next Tab leave the editor
`frontend/app/adventure.tsx`	The editor instruction now says how to leave it
`frontend/tests/keyboard.spec.ts`	New: 7 keyboard-only tests
`frontend/tests/accessibility.spec.ts`	New: axe-core audit, 8 page states at 2 widths, results after a pass and an error, and a self-test
`frontend/tests/fixtures/unit1-page.snapshot.json`	The one changed sentence, in the 20 places it appears
`frontend/playwright.config.ts`	`PW_BROWSER` chooses the browser; Chromium by default
`frontend/package.json`, `package-lock.json`	`axe-core` 4.14.0, exact, dev only
`.github/workflows/cross-browser.yml`	New: Firefox and WebKit, non-blocking
`docs/plan/screen-reader-test-sheet.md`	New: 14 manual tests
`docs/run-log.md`	This stage

3. What was found

Finding	How
A keyboard user could not reach "Run & check challenge", "Reset code", "Show a nudge" or "Back". Tab in the editor always typed spaces, and Shift + Tab only goes backwards, so the controls after the editor could be reached only by going all the way round the browser	Pressed Tab from the top of card 5 and recorded every stop. Earlier tests called `focus()` on the button, which jumps over the problem
axe-core found **no** WCAG 2.0 to 2.2 A or AA problems on the pages audited	`accessibility.spec.ts`
The editor's focus ring is on the box around it, not the text area	Reading `style.css`. It is visible; the test now looks in the right place

4. Decisions and why

Decision	Options considered	Reason	Who decided
Esc, then Tab, leaves the editor	Ctrl + M (as in GitHub), no Tab-to-indent at all	Esc is the common pattern in code editors and the easiest to say in one sentence. Tab still indents, which beginners need	Implementer
The release lasts one key	Until the editor loses focus	An accidental Esc can't throw a learner out of the editor mid-line	Implementer
Change one learner-facing sentence	Leave the instruction as it was	It would no longer be true or complete	Implementer (**the unit 1 snapshot changes in exactly that sentence**)
`axe-core` as a dev dependency, pinned exactly	Loading it from a CDN in tests	Pinned and reviewed like any dependency; never sent to learners	**Lucas**
Cross-browser weekly, by hand, and on PRs that change its workflow	On every push	Each browser takes about as long as the Chromium run, and a failing report job looks like a failing PR	Implementer
Scan with reduced motion on	Scan mid-animation	Handoff 32: scans during a fade-in report colours no learner sees once it settles	Implementer

5. Bugs found and how they were fixed

The editor trapped forward Tab (section 3). Fixed in `code-editor.tsx`: Esc sets a flag, and the next key clears it; only a Tab straight after Esc moves on. Leaving the editor also clears it.

6. How it was tested

Command	Result
`npx tsc --noEmit` and `npm run build`	OK
`CI=1 npx playwright test tests/accessibility.spec.ts tests/keyboard.spec.ts`	**25 passed**
`CI=1 npx playwright test tests/unit1-unchanged.spec.ts`	**1 passed**, with the reworded sentence
`backend/tests/test_ci_pins.py`, which also checks the new workflow	**4 passed**
The full suites	**In CI on this PR.** The full browser suite has not been reliable on this Mac (handoff 35 section 8), so I am relying on CI for it

Mocks: the API answers are recorded fixtures. Python, the worker and the pages are real. The audit's self-test adds a nameless button and checks axe reports it, so the audit cannot pass because axe failed to load.

Deliberate-break controls (each undone; files matched saved copies byte for byte afterwards)

What I broke	Tests that failed
B1. Esc does nothing (the old behaviour)	**3**
B2. Esc never wears off	**1**
B3. No focus ring on the editor	**1**
B4. The editor's label removed	**2** (axe `label`, at both widths)

7. Known limitations

- **axe finds only some problems,** roughly a third of real ones by common estimates. It is no substitute for the manual sheet.
- **Not audited:** units 2 and 3 lesson pages, the testers page and the parent pages. They can be added the same way.
- **The cross-browser workflow has run only on this PR.** Expect failures at first, especially in WebKit with Pyodide. That is what non-blocking is for: read the report, then decide what to fix.

- **The manual sheet has not been run.**

8. Live checks for you

After deploying: open a challenge, click into the editor, press Tab (four spaces), then Esc and Tab. Focus should go to "Run & check challenge".

9. Questions for you

1. **Who runs the screen reader sheet, and when?** *Recommended: VoiceOver on a Mac first, as it is free and built in, before more learners join.*
2. **When should Firefox and WebKit become blocking?** *Recommended: after four clean weekly reports in a row.*

10. For the docs sync

Where	What
Backlog	QA-01 gets a note: QA-01a built (#PR); the manual part waits for people. QA-03 `in_review`
Stage table	"42. Accessibility and browsers | QA-01a, QA-03 | in_review (#PR, handoff 42)"
Decision log	Esc to leave the editor; `axe-core` added as a dev dependency with Lucas's approval

11. Next concrete action

Review and merge. Then try the live check in section 8.

12. Removing the worktree safely

From the **main** folder, after this PR is merged:

```
cd /Users/DEV/Projects/python-academy
git -C ../python-academy-qa status --short   # expect nothing
git worktree remove ../python-academy-qa
git worktree prune
git branch -d qa/accessibility-browsers
```